

Graph coverage criteria: Applied to test code

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

Graph coverage criteria: Overview

- We look at graphs as structures and look at the following two coverage criteria over graphs.
  - Structural coverage criteria.
  - Data flow coverage criteria.
- Later, we consider software artifacts (code, design elements and requirements) modelled as graphs and see how these coverage criteria apply to them.
- Focus of this lecture: Using graph coverage to test source code.

NPTEL

Outline

We will look at the following modules towards understanding how graph coverage criteria can be used to test code.

- Modelling control flow in code as graphs.
  - Modelling branching, looping etc. in code as graphs.
  - Modelling design elements (function calls, class inheritance etc.) as graphs. (For later lectures). This is also called as a call graph.
- Using structural coverage criteria to test control flow in code.
- Traditional structural testing terminologies and its relation to structural coverage. (Next lecture).
- Using data flow coverage criteria to test data flow in code. (Next lecture).

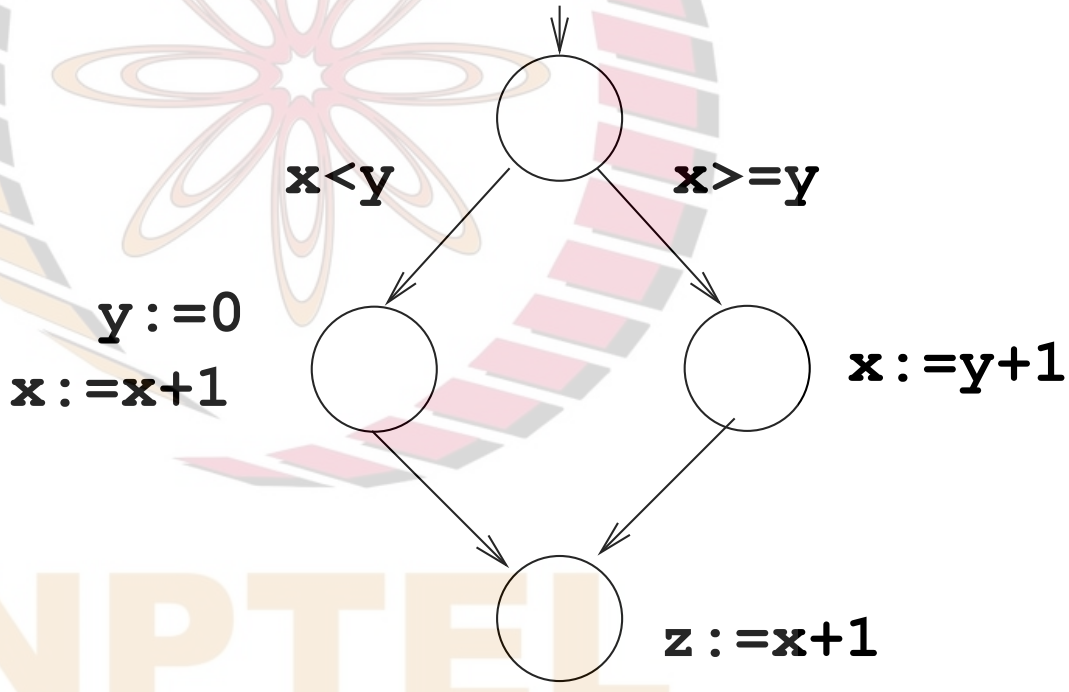
Control flow graphs for code

- A Control Flow Graph (CFG) models all executions of a method by describing control structures.
- Nodes: Statements or sequences of statements (basic blocks).
- Edges: Transfers of control.
- Basic Block: A sequence of statements such that if the first statement is executed, all statements will be (no branches).
- CFGs are sometimes annotated with extra information to model data:
  - Branch predicates.
  - Defs and/or uses.

NPTEL

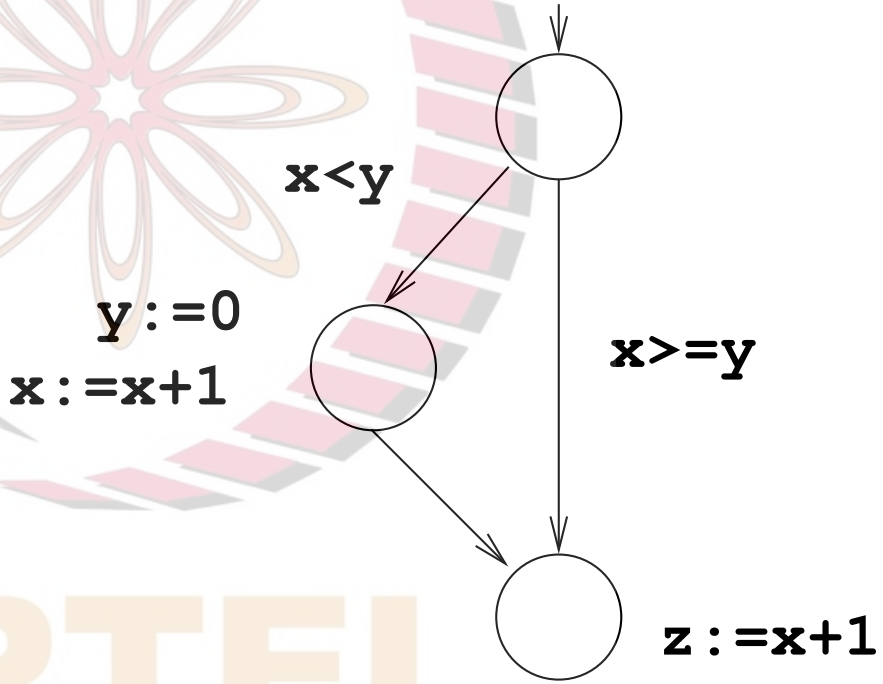
CFG: If statement

```
if (x<y)
{
   y:=0;
   x:=x+1;
}
else
{
   x:=y+1;
}
z:=x+1;
```



CFG: If statement

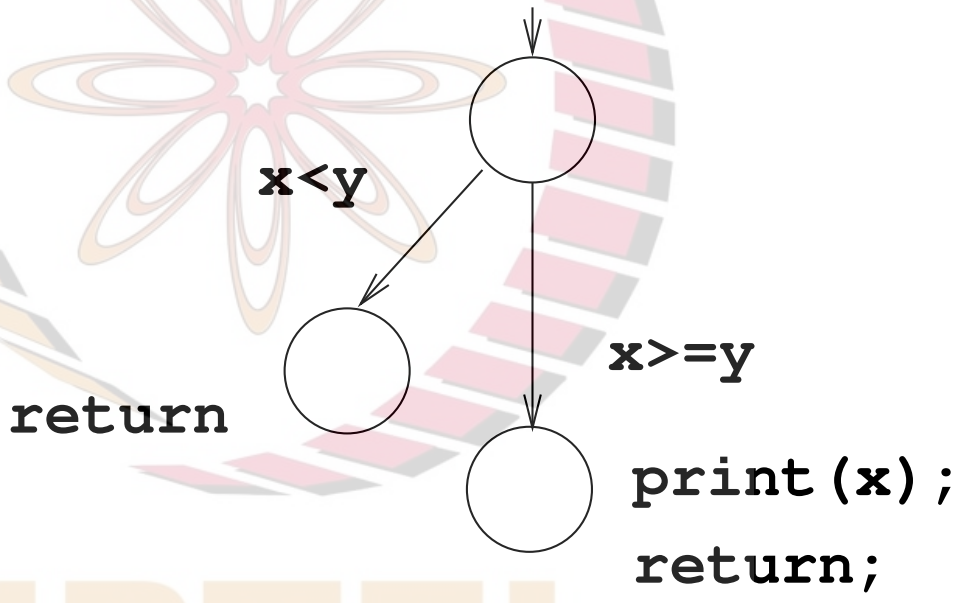
```
if (x<y)
{
   y:=0;
   x:=x+1;
}
z:=x+1;
```



NPTEL

CFG: If statement with return

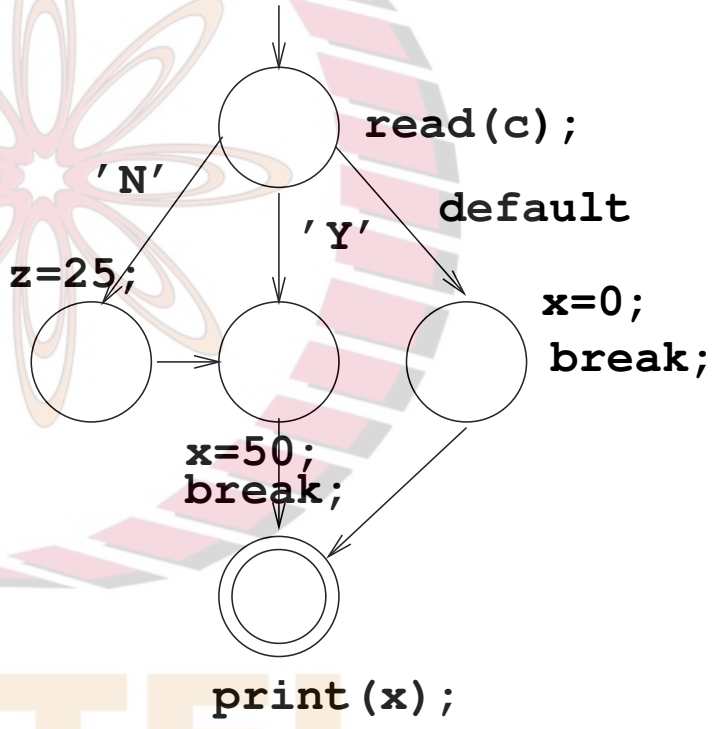
```
if (x<y)
{
  return
}
print(x);
return;
```



Note: There are two nodes corresponding to the two `return` statements.

CFG: Switch-case

```
read(c);
switch(c)
{
 case 'N':z=25;
 case 'Y':{x=50;
           break;}
 default:{x=0;
          break;}
}
print(x);
```



Note: case 'N' without a break statement leads to case 'Y'. It is not so for the other two cases.

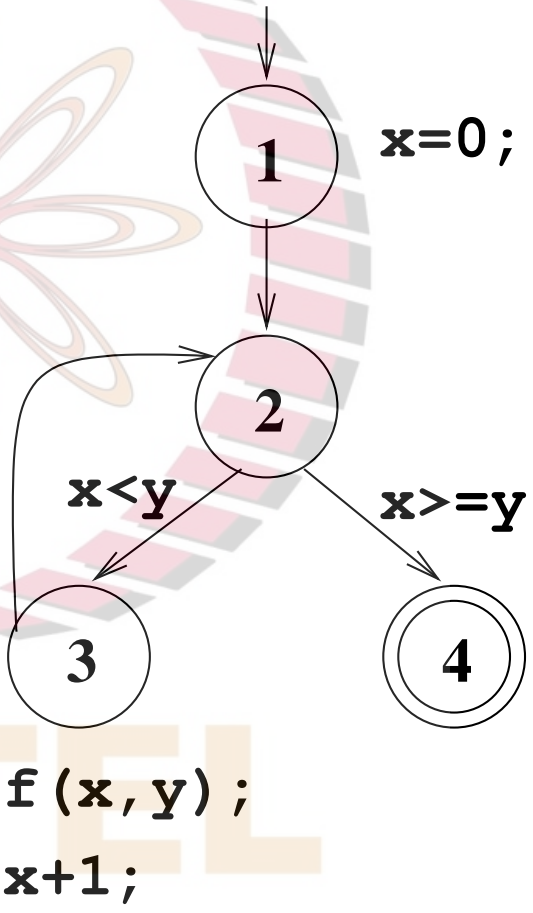
CFG: Loops

- There could be various kinds of loops: while, for, do-while etc.
- To accurately represent the possible branches out of a loop, the CFG for loops need *extra* nodes to be added.

NPTEL

CFG: While loop

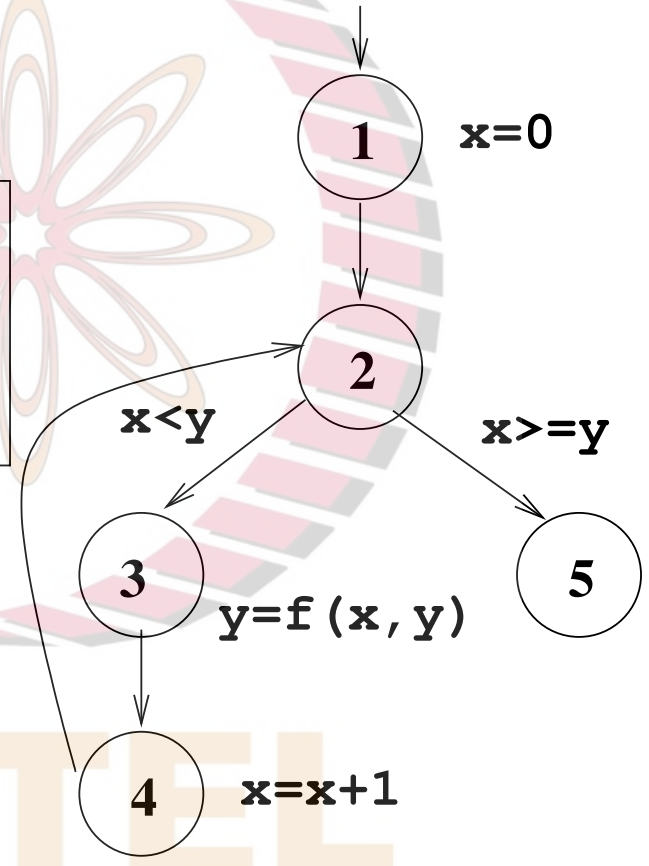
```
x=0;
while(x<y)
{
 y=f(x,y);
 x=x+1;
}
```



Note: Node 2 in the graph above is a dummy node.

CFG: For loop

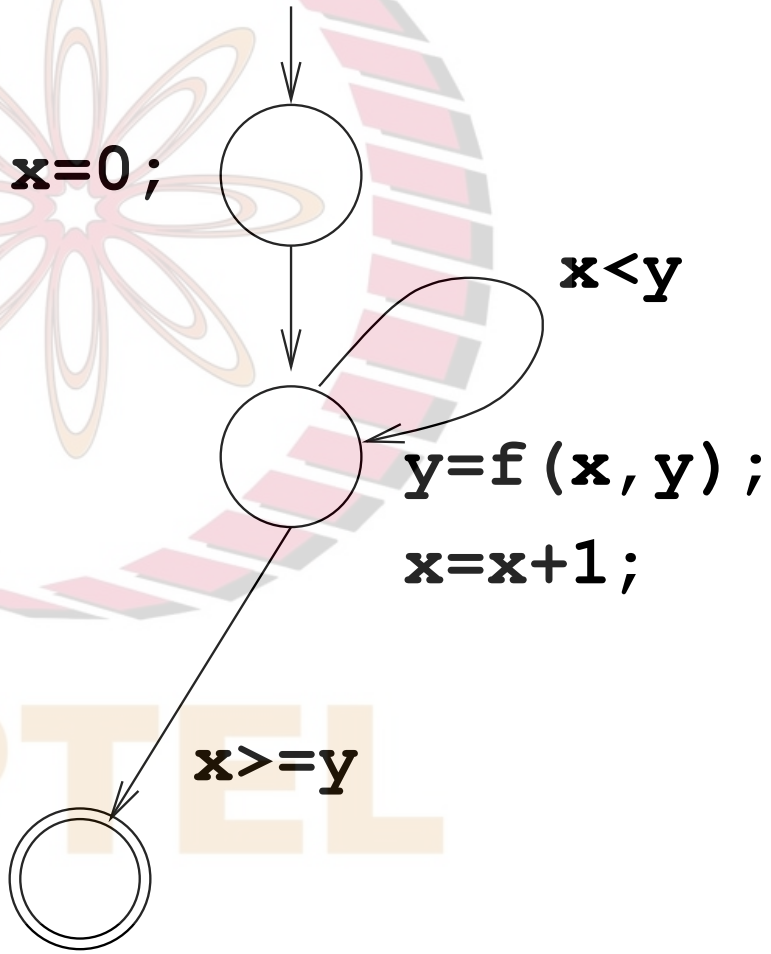
```
for(x=0;x<y;x++)
{
 y=f(x,y);
}
```



Note: Node 1 implicitly initializes the loop, and node 4 implicitly increments the loop.

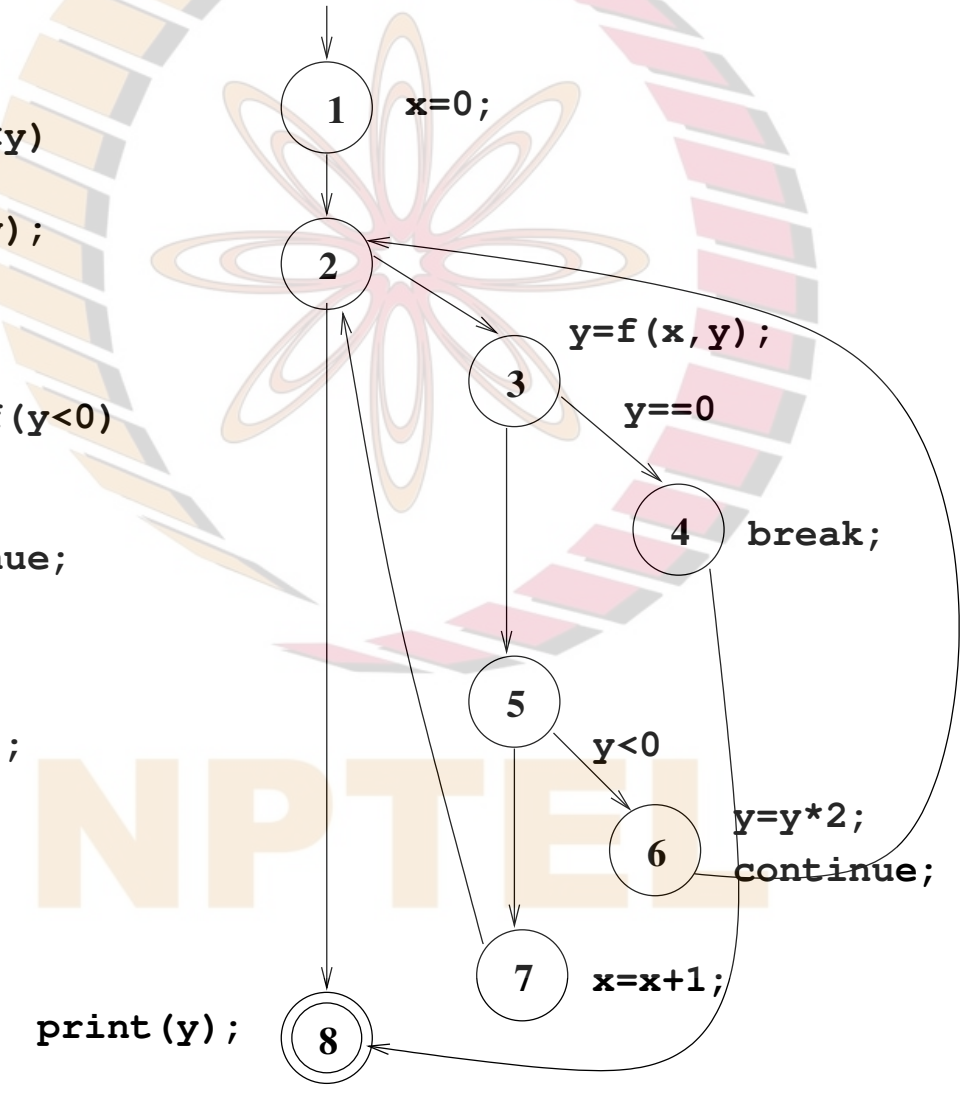
CFG: Do while loop

```
x=0;
do
{
 y=f(x,y);
 x=x+1;
}while(x<y);
print(y);
```



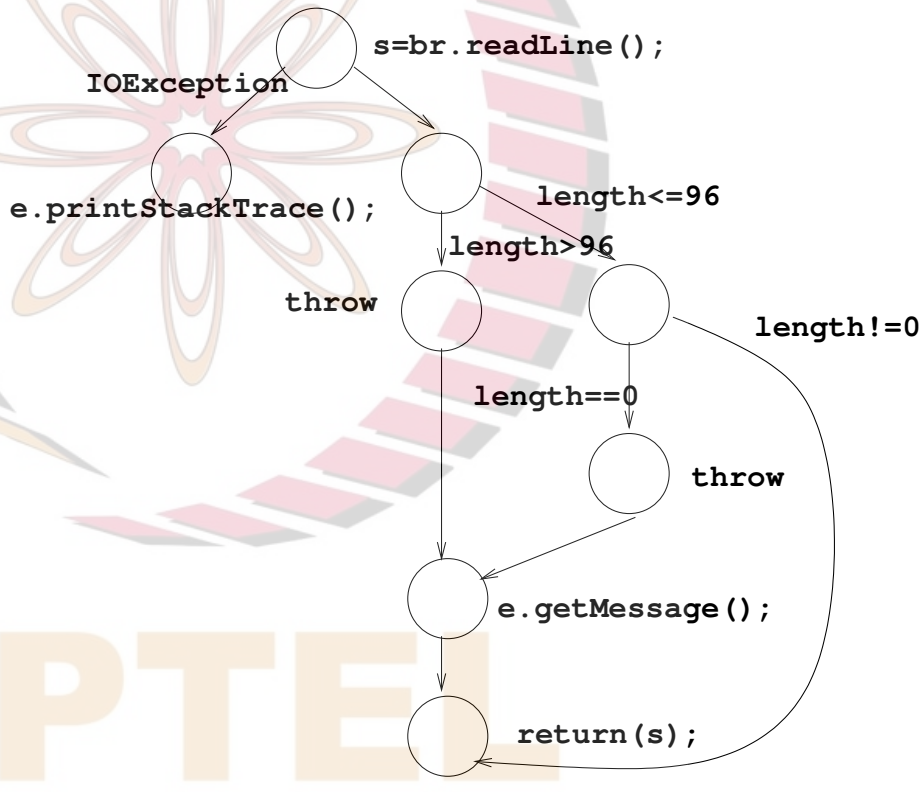
CFG: While loop with break and continue

```
x=0;
while(x<y)
{
 y=f(x,y);
if(y==0)
{
 break;
}else if(y<0)
 {
  y=y*2;
  continue;
 }
 x=x+1;
}
print(y);
```



CFG: Exceptions (try-catch)

```
try
{
 s=br.readLine();
 if(s.length()>96)
  throw new Exception
   ("too long");
 if(s.length()==0)
  throw new Exception
   ("too short");
}(catch IOException e){
 e.printStackTrace();
}(catch Exception e){
  e.getMessage();
}
return(s);
```

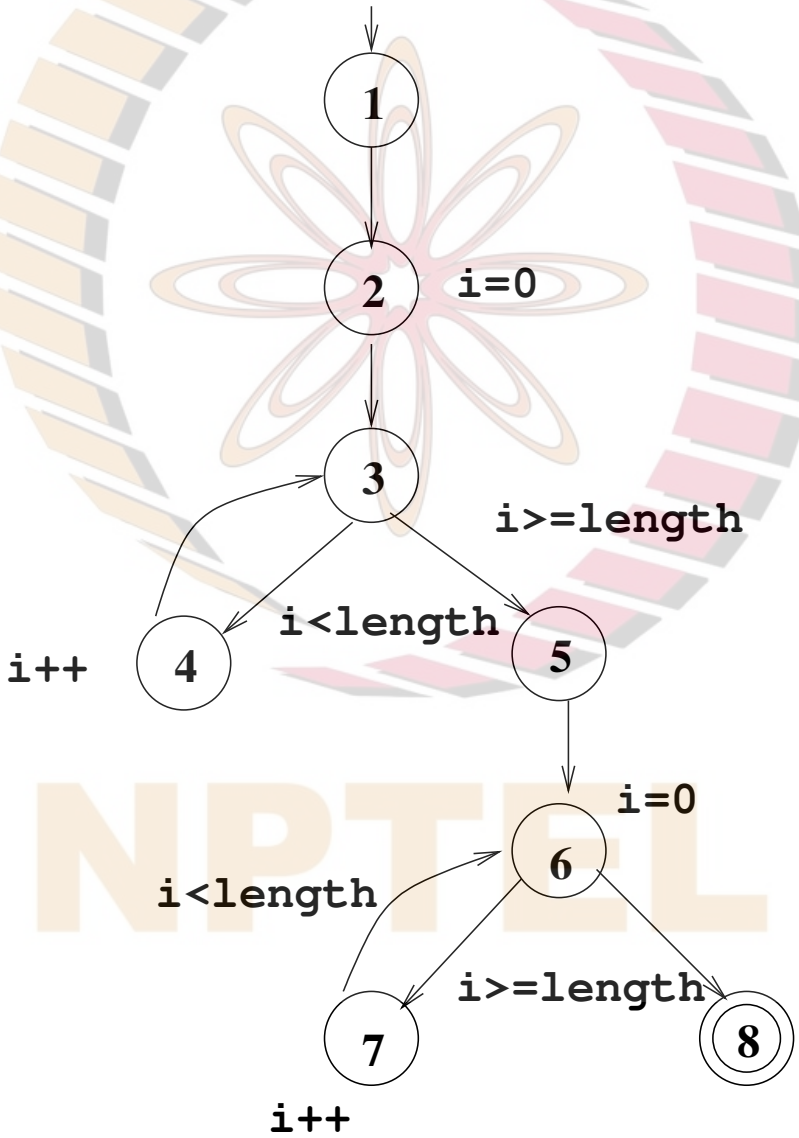


NPTEL

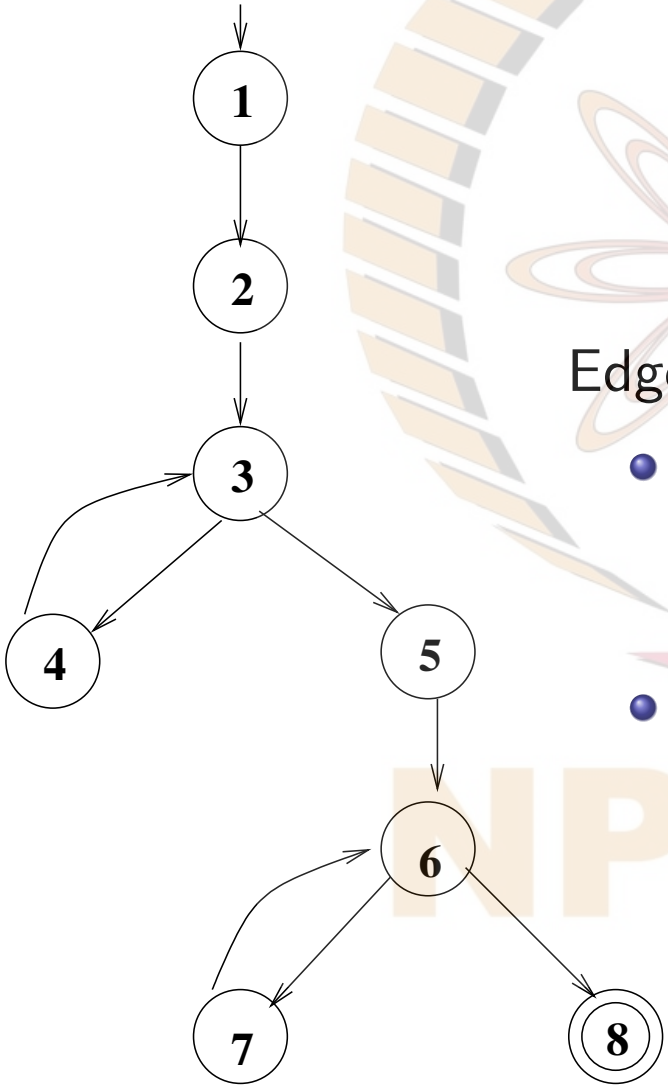
Example program: Statistics

```
public static void computeStats (int [] numbers)
{  int length = numbers.length;
   double med, var, sd, mean, sum, varsum;
   sum = 0.0;
   for(int i=0; i<length; i++)
   {  sum += numbers[i]; }
   med=numbers[length/2];
   mean=sum/(double)length;
   varsum = 0.0;
   for(int i=0; i<length; i++)
   {  varsum = varsum+((numbers[i]-mean)*(numbers[i]-mean)); }
   var = varsum/(length-1);
   sd = Math.sqrt(var);
   System.out.println ("length:" + length);
   System.out.println ("mean:" + mean);
   System.out.println ("median:" + med);
   System.out.println ("variance:" + var);
   System.out.println ("standard deviation:" + sd);
}
```

CFG for Statistics program



Edge coverage for Statistics program

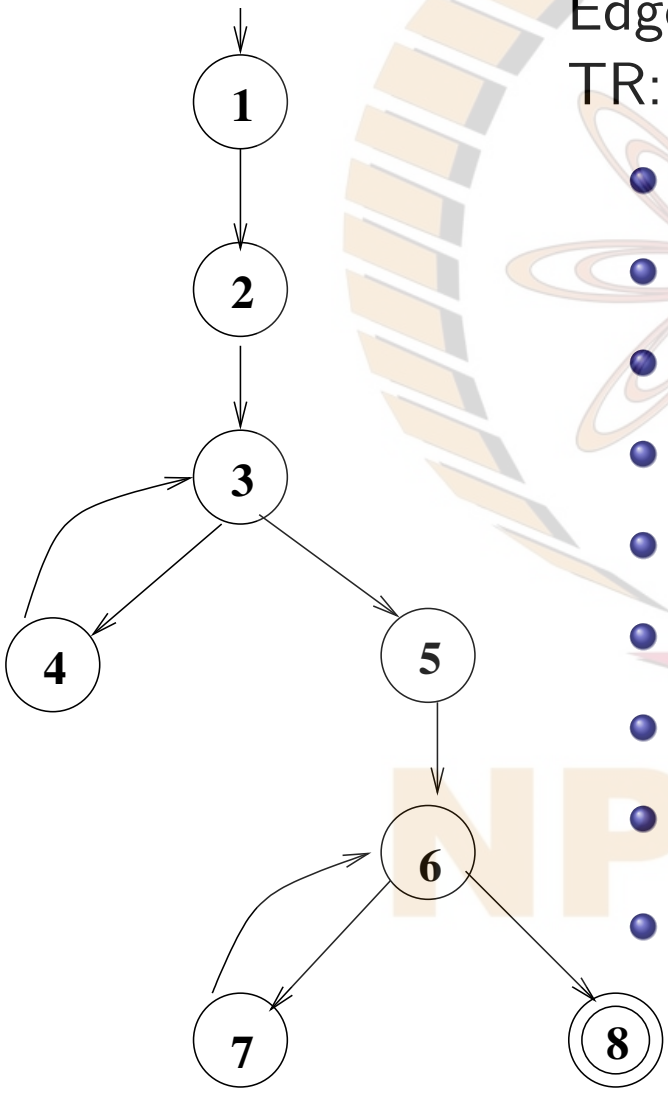


Edge coverage:

- TR: {[1,2],[2,3],[3,4],[4,3],[3,5],[5,6],[6,7],[7,6],[7,8]}.
- Test path: [1,2,3,4,3,5,6,7,6,8]

NPTEL

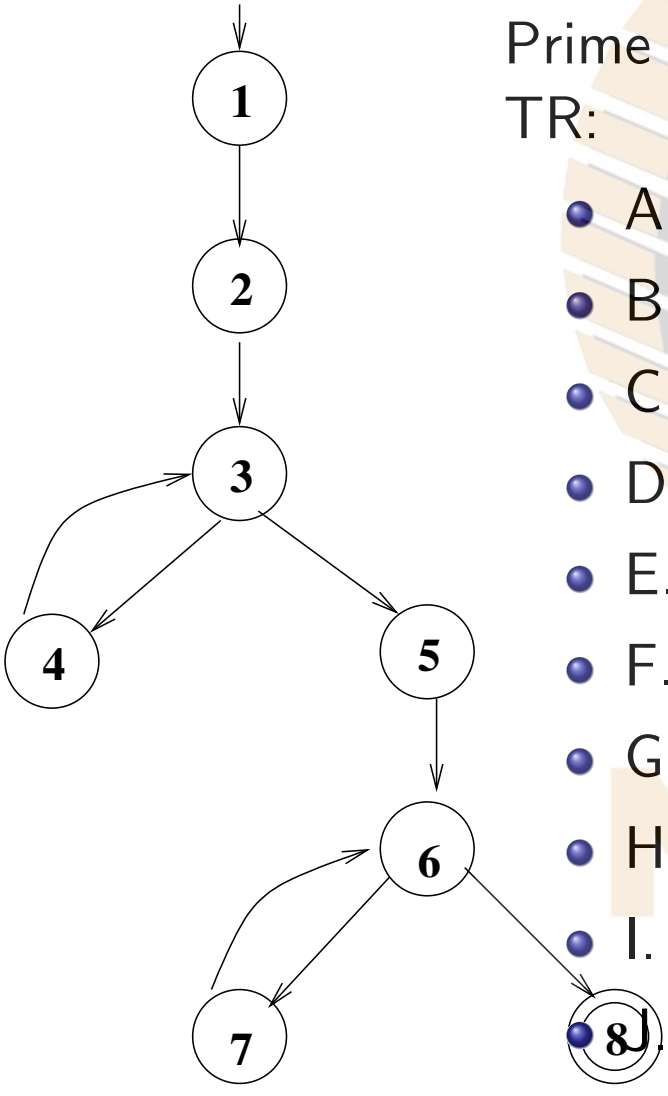
Edge pair coverage for Statistics program



Edge pair coverage:
TR:

- { A. [1, 2, 3]
- B. [2,3,4], C. [2,3,5]
- D. [3,4,3], E. [3,5,6]
- F. [4,3,5], K. [4,3,4]
- G. [5,6,7], H. [5,6,8]
- I. [6,7,6]
- J. [7,6,8]
- L. [7,6,7]}.
- Test paths: [1,2,3,4,3,5,6,7,6,8]
  ii. [1,2,3,5,6,8]
  iii. [1,2,3,4,3,4,3,5,6,7,6,7,6,8]

Prime path coverage for Statistics program



Prime path coverage:

TR:

- A. [3,4,3]
- B. [4,3,4]
- C. [7,6,7]
- D. [7,6,8]
- E. [6,7,6]
- F. [1,2,3,4]
- G. [4,3,5,6,7]
- H. [4,3,5,6,8]
- I. [1,2,3,5,6,7]
- J. [1,2,3,5,6,8]

Test paths:

- i. [1,2,3,4,3,5,6,7,6,8]
- ii. [1,2,3,4,3,4,3,5,6,7,6,7,6,8]
- iii. [1,2,3,4,3,5,6,8]
- iv. [1,2,3,5,6,7,6,8]
- v. [1,2,3,5,6,8]

Credits

Part of the material used in these slides are derived from the presentations of the book Introduction to Software Testing, by Paul Ammann and Jeff Offutt.

NPTEL